

1. Embedding PHP in HTML

- PHP code is inserted into HTML using `<?php ... ?>`.
- The file must have the extension `.php`.
- PHP interprets the code and generates corresponding HTML to send to the client.
- Syntax errors (e.g., lexical or syntactic) will stop execution and display an error based on the `php.ini` configuration.

Minimal PHP Example

PHP

```
<?php  
echo "This is PHP";  
?>
```

HTML Example

Server-side code:

HTML

```
<HTML>  
<HEAD><TITLE>Test</TITLE></HEAD>  
<BODY>  
<?php echo("2nd Y Prepa"); ?>  
</BODY>  
</HTML>
```

Client-side output:

HTML

```
<HTML>  
<HEAD><TITLE>Test</TITLE></HEAD>  
<BODY>  
2nd Y Prepa  
</BODY>  
</HTML>
```

2. Variables in PHP

- Variables are prefixed with `$` (e.g., `$x`, `$name`).
- No declaration is required; variables are created on assignment.
- PHP is not strongly typed; the type is determined by the context.
- **Variable Deletion:** Use `unset($variable)` to free memory.

Supported Types

- **Scalars:** `boolean`, `int`, `float`, `string`.
- **Compounds:** Arrays, Objects.
- **Special:** `NULL`.

Boolean Examples

- `TRUE` is any non-zero value, e.g., `-1`, `"abc"`.
- `FALSE` includes `0`, `0.0`, `""`, `"0"`, `NULL`, empty array, or object.

Integer Examples

- Decimal: `$x = 141; $y = -141;`
- Octal: `$x = 0215;`
- Hexadecimal: `$x = 0x8D;`

Real and String Examples

- Real Numbers: `$pi = 3.14;`
- Strings: `$str1 = "Welcome"; $str2 = 'PHP';`

3. Arrays in PHP

- **Definition:** Arrays are associative, with keys (indices) mapping to values.
- **Keys:** Can be integers or strings.
- **Values:** Any PHP data type.

Creating Arrays

1. **Key-Value Assignment:**

PHP

```
$tab['name'] = "Mohamed";  
$tab[3] = "Ali";
```

2. Using `array()` Function:

PHP

```
$tab = array('name' => 'Mohamed', 3 => 'Ali');  
$tab = array("value1", "value2"); // Indexed automatically
```

Multi-Dimensional Arrays

PHP

```
$matrix['name'][1] = "Mohamed";  
$matrix['name'][2] = "Ali";
```

Unset Elements

PHP

```
unset($tab['key']); // Removes the element with key 'key'
```

4. References in PHP

- Use the `&` symbol to make variables point to the same memory location.

Example

PHP

```
$a = 1;
$b =& $a; // $b references $a

echo $a; // Outputs: 1
echo $b; // Outputs: 1

$a = 2;
echo $b; // Outputs: 2
```

5. Dynamic Variables in PHP

- Variable names can themselves be dynamic using `${}` or `$$`.

Examples

1. Simple Dynamic Variable:

PHP

```
$x = "y";
$$x = 1; // $y = 1
echo $y; // Outputs: 1
```

2. Dynamic Arrays:

PHP

```
$z = "arrayName";
${$z}[1] = 2; // $arrayName[1] = 2
echo $arrayName[1]; // Outputs: 2
```

6. Constants

1. Predefined Environment Variables:

- Begin with `$_` and are built-in by PHP.
- Examples:
 - `$_SERVER["SERVER_NAME"]`: Server name.
 - `$_SERVER["REMOTE_ADDR"]`: Client's IP address.

2. User-Defined Constants:

- Use `define("CONSTANT_NAME", "value");`.
- Constants cannot be changed after definition.

Example

PHP

```
define("PI", "3.14");  
echo PI; // Outputs: 3.14
```

Table for Environment Variables (`$_SERVER`)

Variable/Function	Description	Example
<code>\$_SERVER['PHP_SELF']</code>	Filename of the currently executing script.	<code>echo \$_SERVER['PHP_SELF'];</code> → <code>/folder/index.php</code>
<code>\$_SERVER['SERVER_NAME']</code>	Server's name (domain name or localhost).	<code>echo</code> <code>\$_SERVER['SERVER_NAME'];</code> → <code>localhost</code>
<code>\$_SERVER['REQUEST_METHOD']</code>	The method used for the request (e.g., GET, POST).	<code>echo</code> <code>\$_SERVER['REQUEST_METHOD'];</code> → <code>GET</code> or <code>POST</code>

Variable/Function	Description	Example
<code>\$_SERVER['QUERY_STRING']</code>	The query string from the URL (everything after the <code>?</code>).	<code>echo \$_SERVER['QUERY_STRING'];</code> → <code>id=1&name=John</code>
<code>\$_SERVER['HTTP_USER_AGENT']</code>	The user agent string (browser, OS info).	<code>echo \$_SERVER['HTTP_USER_AGENT'];</code> → <code>Mozilla/5.0 (Windows NT)</code>
<code>\$_SERVER['SERVER_ADDR']</code>	IP address of the server.	<code>echo \$_SERVER['SERVER_ADDR'];</code> → <code>192.168.1.1</code>
<code>\$_SERVER['DOCUMENT_ROOT']</code>	The document root directory on the server.	<code>echo \$_SERVER['DOCUMENT_ROOT'];</code> → <code>/var/www/html</code>
<code>\$_SERVER['REMOTE_ADDR']</code>	IP address of the client (user).	<code>echo \$_SERVER['REMOTE_ADDR'];</code> → <code>192.168.1.2</code>
<code>\$_SERVER['REQUEST_URI']</code>	The full URI of the request (after the domain name).	<code>echo \$_SERVER['REQUEST_URI'];</code> → <code>/index.php?id=1</code>

Form Data

1. Form Fields:

- Data from forms is collected based on field names (`field_name`).
- Data can be accessed using `$_POST` or `$_GET` depending on the form submission method.
- Example:

```
<form method="POST">
  <input type="text" name="username">
</form>
```

- Accessed using `$_POST['username']`.

2. Checking Data:

- Use `isset()` to check if the data has been sent.
- Example:

```
if (isset($_POST['username'])) {
    // Process data
}
```

PHP Operators

1. Assignment Operator

- Used to assign values to variables.
- Example:

```
$a = 2; // Assigns value 2 to $a
$b = "Hello"; // Assigns string "Hello" to $b
```

2. Arithmetic Operators

- `+`: Addition
- `-`: Subtraction
- `*`: Multiplication
- `/`: Division (real result)
- `%`: Modulo (remainder of division)

Examples:

PHP

```
$sum = 10 + 5; // 15
$product = 10 * 2; // 20
$remainder = 10 % 3; // 1
```

3. String Operator

- `.`: Concatenates two strings.
- Example:

PHP

```
$greeting = "Hello" . " " . "World"; // "Hello World"
```

4. Comparison Operators

- `==`: Equal to
- `===`: Equal and same type
- `!=`: Not equal
- `>`: Greater than
- `>=`: Greater than or equal
- `<`: Less than
- `<=`: Less than or equal

Example:

PHP

```
if ($a == $b) { // checks if $a is equal to $b
    echo "Equal";
}
```

5. Logical Operators

- `&&`: AND
- `||`: OR
- `xor`: Exclusive OR
- `!`: NOT

Example:

PHP

```
if ($a > 5 && $b < 10) {  
    // code  
}
```

6. Bitwise Operators

- **&**: AND
- **|**: OR
- **^**: XOR (exclusive OR)
- **~**: NOT
- **<<**: Shift left
- **>>**: Shift right

Example:

PHP

```
$result = $a & $b; // Bitwise AND of $a and $b
```

Abbreviated Expressions

- **Chained Assignment:** Assigns a value to multiple variables at once.

PHP

```
$x = $y = 2; // $x and $y both get 2
```

- **Increment and Decrement:**
 - **++**: Increment by 1
 - **--**: Decrement by 1

```
$x++; // Increments $x by 1  
--$x; // Decrements $x by 1
```

- **Post and Pre-Increment/Decrement:**

- `++$x`: Increment before using the value.
- `$x++`: Use the value before incrementing.

```
$y = $x++; // $y gets current $x, then $x is incremented  
$y = ++$x; // $x is incremented first, then $y gets the  
value
```

- **Concatenation Assignment:** Concatenates and assigns a string.

```
$x .= "World"; // Concatenates "World" to $x
```

Converting Variables

1. Automatic Type Conversion:

- PHP automatically converts variables to the correct type based on the operation.
- Example:

```
$x = 1 + 2.5; // $x will be a float with value 3.5  
$x = 1 + "2.5"; // $x will be 3.5
```

2. Type Casting:

- You can force a variable to be a certain type using `settype()` or explicit casting.
- **Using `settype()`:**

PHP

```
$x = 3.6;  
settype($x, "int"); // $x will be 3
```

- **Using explicit casting:**

PHP

```
$a = (int) (8 / 3); // $a will be 2
```

3. Check Type:

- Use functions like `gettype()` or `is_type()` to check the variable type.
- Example:

PHP

```
$t = gettype($var); // returns type as a string  
if (is_int($var)) { // checks if $var is an integer  
    echo "Integer";  
}
```

Character Strings

1. Single Quotes (`'`):

- Display text as is; variables won't be evaluated.
- Escape characters: `\'` and `\\`.
- Example:

```
$x = 2;  
echo 'Value of x is $x'; // Outputs: Value of x is $x
```

2. Double Quotes ("):

- Variables are evaluated, and escape sequences like `\n` for newlines are processed.
- Example:

```
$x = 2;  
echo "The value of x is $x"; // Outputs: The value of x  
is 2  
echo "Line break here:\n New line"; // Outputs: Line  
break here:  
                                     // New line
```

3. Escaping with Backslashes:

- Use backslashes to escape special characters inside strings.
- Example:

```
echo "This is a \"quote\""; // Outputs: This is a  
"quote"
```

Strings Management

1. Getting Length of a String:

- Use `strlen()` to get the length of a string.
- Example:

```
$len = strlen("Hello"); // $len will be 5
```

2. Comparing Strings:

- Use `strcmp()` to compare two strings.
- Example:

```
$result = strcmp("apple", "banana"); // Returns a  
negative value (because "apple" < "banana")
```

3. Accessing Characters in a String:

- Use index to access individual characters in a string.
- Example:

```
$str = "Hello";  
$char = $str[0]; // $char will be 'H'
```

4. Replacing Parts of a String:

- Use `str_replace()` to replace occurrences of a substring.
- Example:

```
$new_str = str_replace("apple", "orange", "I like  
apple"); // Outputs: I like orange
```

5. Adding Slashes to Special Characters:

- Use `addslashes()` to add backslashes before special characters.
- Example:

```
$escaped = addslashes("O'Reilly"); // Outputs: O\'Reilly
```

6. Removing Slashes from Special Characters:

- Use `stripslashes()` to remove slashes.
- Example:

```
$unesaped = stripslashes("O\\\'Reilly"); // Outputs:  
O'Reilly
```

Regular Expressions (RegExp)

1. Anchors:

- `^`: Indicates the beginning of a string.
 - Example: `^start` → matches "start" at the beginning of a string.
- `$`: Indicates the end of a string.
 - Example: `end$` → matches "end" at the end of a string.
- `^string$` → matches the exact string "string".

2. Character Sets:

- `[a]` or `a`: Matches exactly the character `a`.
- `[abc]`: Matches either `a`, `b`, or `c`.
- `[ac]`: Matches either `a` or `c`.
- `[0-9]`: Matches any numeric character (0 to 9).
- `[az]`: Matches any lowercase alphabetic character.
- `[AZ]`: Matches any uppercase alphabetic character.
- `[a-zA-Z0-9]`: Matches any alphanumeric character (letters and digits).
- `.`: Matches any character except newlines.

3. Quantifiers:

- `{n}`: Matches exactly `n` occurrences of the preceding character.
 - Example: `abc{2}` → matches "abcc".
- `{n,}`: Matches at least `n` occurrences of the preceding character.
 - Example: `abc{2,}` → matches "abcc", "abccc", etc.

- **{n,m}**: Matches between **n** and **m** occurrences of the preceding character.
 - Example: **a{2,4}** → matches "aa", "aaa", "aaaa".
- *****: Matches 0 or more occurrences.
- **+**: Matches 1 or more occurrences.
- **?**: Matches exactly one occurrence or none.

4. Example Patterns:

- **Date format (dd-mm-yyyy)**: **^[0-9]{2}\-[0-9]{2}\-[0-9]{4}\$**.
- **Identifier (alphanumeric)**: **^[a-zA-Z][a-zA-Z0-9]*\$**.

RegExp Management

1. **preg_match("reg_exp", \$ch):**

- **Purpose**: Checks if a regular expression matches the string **\$ch**.
- **Returns**: **TRUE** or **FALSE**.
- **Example**:

PHP

```
preg_match("#^[A-Za-z0-9_.-]{4,20}$#", $user);
```

This checks if **\$user** contains between 4 to 20 alphanumeric characters, dots, underscores, or hyphens.

2. **preg_replace("reg_exp", \$ch1, \$ch2):**

- **Purpose**: Replaces occurrences of a string matching the regular expression in **\$ch2** with the string defined in **\$ch1**.
- **Example**:

PHP

```
preg_replace("#abc#", "xyz", "abc abc abc");  
// Output: "xyz xyz xyz"
```

3. **preg_split("reg_exp", \$ch):**

- **Purpose**: Splits the string **\$ch** into multiple substrings based on the regular expression, storing them in the **\$tab** array.

- **Example:**

PHP

```
$tab = preg_split("#,#", "apple,orange,banana");  
// $tab = ["apple", "orange", "banana"]
```

Array Management

1. Sorting Arrays:

- **sort(\$tab)**: Sorts an array by its values in ascending order.
 - Example:

PHP

```
sort($arr); // $arr = [1, 2, 3, 4]
```

- **rsort(\$tab)**: Sorts an array by its values in descending order.
 - Example:

PHP

```
rsort($arr); // $arr = [4, 3, 2, 1]
```

2. Sorting by Keys:

- **ksort(\$tab)**: Sorts an array by its keys in ascending order.
 - Example:

PHP

```
ksort($arr); // Sorts by the keys
```

- **krsort(\$tab)**: Sorts an array by its keys in descending order.
 - Example:


```
krsort($arr); // Sorts by the keys in reverse order
```

3. Pointer Functions:

- **reset(\$tab)**: Resets the array pointer to the first element.
 - Example:

PHP

```
reset($arr); // Pointer goes to the first element.
```

- **end(\$tab)**: Moves the array pointer to the last element.
 - Example:

PHP

```
end($arr); // Pointer goes to the last element.
```

4. Array Size:

- **sizeof(\$tab)**: Returns the number of elements in the array **\$tab**.
 - Example:

PHP

```
$size = sizeof($arr); // Returns the number of  
elements in the array
```

Conditional Statements

1. Basic **if** Statement:

- **Syntax:**

```
if (condition) {  
    // Code to execute  
}
```

- **Example:**

```
if ($x > 0) {  
    $x--;  
}
```

2. **if...else Statement:**

- **Syntax:**

```
if (condition) {  
    // Code to execute if condition is true  
} else {  
    // Code to execute if condition is false  
}
```

- **Example:**

```
if ($x > 0) {  
    $x--;  
} else {  
    $x++;  
}
```

3. **if...elseif...else Statement:**

- **Syntax:**

```
if (cond1) {  
    // Block 1  
} elseif (cond2) {  
    // Block 2  
} else {  
    // Default Block  
}
```

- **Example:**

```
if ($x > 0) {  
    $x--;  
} elseif ($x == 0) {  
    $x++;  
} else {  
    $x *= 2;  
}
```

4. Ternary Operator (Abbreviated Conditional):

- **Syntax:**

```
$var = (condition) ? value1 : value2;
```

- **Example:**

```
$status = ($x > 0) ? "Positive" : "Negative";
```

5. **switch** Statement:

- **Syntax:**

```
switch ($var) {  
    case value1:  
        // Code for case 1  
        break;  
    case value2:  
        // Code for case 2  
        break;  
    default:  
        // Default case code  
}
```

- **Example:**

```
switch ($x) {  
    case 1:  
        echo "One";  
        break;  
    case 2:  
        echo "Two";  
        break;  
    default:  
        echo "Unknown";  
}
```

6. Loops:

- **while Loop:** Repeats as long as the condition is true.

```
while ($x < 10) {  
    $x++;  
}
```

- **do...while Loop:** Executes the block of code first, then checks the condition.

```
do {  
    $x++;  
} while ($x < 10);
```

- **for Loop**: Initializes variables, checks the condition, and updates the variables after each iteration.

```
for ($i = 0; $i < 10; $i++) {  
    echo $i;  
}
```

- **foreach Loop**: Loops through each element of an array.

```
foreach ($arr as $val) {  
    echo $val;  
}
```

7. Loop Control:

- **break**: Exits the loop immediately.
- **continue**: Skips the current iteration and proceeds to the next one.

Function Definition

- **Syntax:**

```
function function_name($arg1, $arg2, ..., $argn) {  
    statement_1;  
    statement_2;  
    ...  
    statement_n;  
    return $var;  
}
```

- **Explanation:**

- A function is defined with the keyword **function**, followed by the name of the function, and a list of arguments in parentheses.
- Inside the function, you write the statements that define the logic or action the function will perform.
- The **return** keyword is used to return a value from the function.

Return Statement

- The **return** statement allows the function to send back a value to the calling code.
- It can be placed anywhere inside the function.

Functions Without Arguments

- Functions can be defined without arguments:

```
function function_name() {  
    // statements  
}
```

Functions Without Return Values (Procedure)

- Functions can also be created to perform an action without returning a value.
-

Return Multiple Values

- **Using Arrays:**

- You can return multiple values by using an array:

PHP

```
return array($key_1 => $val_1, ..., $key_n => $val_n);
```

or:

PHP

```
return array($val_1, ..., $val_n);
```

- **Array as a Variable:**

- The variable used to return multiple values must be of type **array**.

Default Settings (Arguments)

- **Default Values for Arguments:**

- Arguments can have default values when defining a function:

PHP

```
function function_name($x, $y, $z = 1) {  
    // statements  
}
```

- Default arguments must always be placed at the end of the argument list.

Calling a Function

- **Function Call Syntax:**

```
function_name(arg1, ..., argn){  
    // function_definition  
}  
$idf_var = function_name(arg1, ..., argn);
```

- The function must be called after it is defined.
 - The arguments passed during the function call can be variables or values.
-

Passing Arguments

- **By Value:**
 - Arguments are passed by value by default.
 - The value of the argument is used within the function, and changes do not affect the original variable.
- **By Reference:**
 - Passing by reference allows the function to modify the original variable.
 - To pass by reference, use **&** before the argument in the function definition:

```
function function_name(&$var) {  
    // statements  
}
```

Examples of Passing Arguments

- **Passing by Value:**


```
function f1($x) {  
    $x++;  
}  
  
$a = 1;  
f1($a);  
echo $a; // Will output 1 (no change to $a)
```

- **Passing by Reference:**

```
function f2(&$y) {  
    $y++;  
}  
  
$a = 1;  
f2($a);  
echo $a; // Will output 2 (value of $a changes)
```

Global and Local Variables

- **Global Variables:**

- A global variable is visible and accessible throughout the entire PHP code.
- Can be accessed using the `$GLOBALS` array or the `global` keyword:

```
global $var;  
$GLOBALS["var"];
```

- **Local Variables:**

- A local variable is visible only inside the function.
-

Examples

- **Local Variable:**

PHP

```
$a = 1;
function f1() {
    echo $a; // Will not display anything because $a is
    local to the main code
}
f1();
```

- **Global Variable (Using `global`):**

PHP

```
$a = 1;
function f2() {
    global $a, $b;
    $b = $a++;
}
f2();
echo $a; // Will print 2 (because $a is incremented inside
f2)
echo $b; // Will print 1 (b is assigned the value of $a
before incrementing)
```

- **Global Variable (Using `$GLOBALS`):**

PHP

```
$a = 1;
function f3() {
    $GLOBALS["b"] = $GLOBALS["a"]++;
}
f3();
echo $a; // Will print 2
echo $b; // Will print 1
```

- **Static Variable:**

- A static variable retains its value across multiple calls to the function.

PHP

```
function f4() {  
    static $a = 2;  
    $a++;  
    echo $a;  
}  
  
f4(); // Will display 3  
echo $a; // Will display 1 (global $a is unaffected)  
f4(); // Will display 4 (static $a is incremented)
```

Predefined Functions

- **Function Example:**

PHP

```
$y = abs($x); // Returns the absolute value of $x
```

- **ceil(\$x)**: Returns the smallest integer greater than or equal to \$x (round up).
- **floor(\$x)**: Returns the largest integer less than or equal to \$x (round down).
- **round(\$x, \$N)**: Rounds \$x to \$N decimal places.
- **fmod(\$x, \$y)**: Returns the remainder of \$x divided by \$y.
- **sqrt(\$x)**: Returns the square root of \$x.
- **pow(\$x, \$n)**: Returns \$x raised to the power of \$n.

Predefined Functions (Extension Modules)

- **Checking Loaded Extensions:**

```
$tab = get_loaded_extensions();
```

- Returns an array of all installed extension modules.

- **Listing Functions of a Module:**

```
$tab = get_extension_funcs($module_name);
```

- Returns an array of function names available in a specific extension module.

Date Management in PHP

- **Unix Timestamp**

- The Unix timestamp represents the number of seconds since 01/01/1970 at 00:00:00 (the Unix epoch).
- PHP's `time()` function returns the current timestamp.
- Example:

```
$t = time(); // Current timestamp  
echo $t; // Outputs current timestamp, e.g., 1675075147
```

- **getdate() Function**

- The `getdate()` function returns an associative array containing various components of the date based on the timestamp passed to it.
- This function provides detailed information such as the weekday, day of the month, month, year, hour, minute, second, and more.
- Example:

```
$t = time(); // Current timestamp
$d = getdate($t); // Get details about the current date

echo $d['weekday']; // Outputs the full weekday name,
e.g., "Friday"
echo $d['mday']; // Outputs the day of the month,
e.g., "28"
echo $d['month']; // Outputs the full month name,
e.g., "January"
echo $d['year']; // Outputs the year, e.g., "2025"
```

- Key elements of the array:
 - `$d['wday']`: Day of the week (0 = Sunday, 1 = Monday, ..., 6 = Saturday).
 - `$d['weekday']`: Full name of the weekday.
 - `$d['mday']`: Day of the month (1 to 31).
 - `$d['mon']`: Month number (1 to 12).
 - `$d['month']`: Full name of the month.
 - `$d['year']`: Year in four digits.
 - `$d['hours']`, `$d['minutes']`, `$d['seconds']`: Time in hours, minutes, and seconds.
 - `$d['yday']`: Day of the year (1 to 366).
- **checkdate() Function**
 - The `checkdate()` function is used to check if a given date is valid.
 - It takes three arguments: the month, day, and year, and returns **TRUE** if the date is valid, **FALSE** if it's invalid.
 - Example:

PHP

```
$b = checkdate(2, 30, 2022); // February 30, 2022 is
not valid
if ($b) {
    echo "The date is valid.";
} else {
    echo "The date is invalid."; // Outputs: "The date
is invalid."
}
```

- Another example:

PHP

```
$b = checkdate(2, 28, 2022); // February 28, 2022 is
valid
if ($b) {
    echo "The date is valid."; // Outputs: "The date is
valid."
} else {
    echo "The date is invalid.";
}
```

File Management in PHP

- **File Existence Test**

- `file_exists($file_name)` checks if a file exists. Returns **TRUE** if the file exists, otherwise **FALSE**.
 - Example:

PHP

```
$b = file_exists('file.txt'); // TRUE if 'file.txt'
exists
```

- **Creating a File**

- `touch($file_name, $t)` creates a file with the name `$file_name` and sets its last modified timestamp to `$t`.
 - Example:

```
$t = time(); // Current timestamp  
touch('newfile.txt', $t); // Creates 'newfile.txt'
```

- **Verifications on Files**

- `is_file($file_name)` checks if it is a regular file.

- Example:

```
$b = is_file('file.txt'); // TRUE if it's a regular  
file
```

- `is_readable($file_name)` checks if the file is readable.

- Example:

```
$b = is_readable('file.txt'); // TRUE if readable
```

- `is_writable($file_name)` checks if the file is writable.

- Example:

```
$b = is_writable('file.txt'); // TRUE if writable
```

- `is_uploaded_file($file_name)` checks if the file was uploaded via HTTP POST.

- Example:

```
$b = is_uploaded_file('file.txt'); // TRUE if  
uploaded via a form
```

- `filetype($file_name)` returns the type of the file (e.g., 'file', 'dir').

- Example:

```
$type = filetype('file.txt'); // Returns file type
```

- **Opening a File**

- `fopen($file_name, $mode)` opens a file in a specified mode and returns a resource variable.

- Example:

```
$f = fopen('file.txt', 'r'); // Opens file in read mode
```

- **File Modes**

- `r`: Open for reading.
- `w`: Open for writing (overwrites content).
- `a`: Open for appending (adds to the end of the file).
- Modes with `b` treat the file as binary (e.g., `rb`, `wb`).

- **Path Parameter**

- Optional `$path` parameter extends file search to subdirectories when set to `TRUE`.

- **Locking a File**

- `flock($f, $n)` locks the file to prevent simultaneous access.

- Example:

```
$b = flock($f, LOCK_EX); // Locks for writing
```

- **Reading from Files**

- `fgets($f, $num_bytes)` reads a line from the file (up to `$num_bytes`).

- Example:

```
$ch = fgets($f, 100); // Reads up to 100 bytes
```


- `fread($f, $num_bytes)` reads a specific number of bytes from the file.
 - Example:

PHP

```
$ch = fread($f, 100); // Reads 100 bytes
```

- `fgetc($f)` reads a single character from the file.
 - Example:

PHP

```
$c = fgetc($f); // Reads a character
```

- **Writing to Files**

- `fputs($f, $ch, $n)` writes the string `$ch` to the file, with an optional `$n` specifying the number of characters to write.
 - Example:

PHP

```
fputs($f, 'Hello, World!', 5); // Writes first 5  
characters
```

- `fwrite($f, $ch, $n)` writes the string `$ch` to the file with an optional `$n` specifying the number of characters to write.
 - Example:

PHP

```
fwrite($f, 'Hello, World!', 5); // Writes first 5  
characters
```

- **End of File**

- `feof($f)` checks if the end of the file has been reached.
 - Example:

PHP

```
$b = feof($f); // TRUE if at end of file
```

- **File Size**

- `filesize($f)` returns the size of the file in bytes.
 - Example:

PHP

```
$size = filesize('file.txt'); // Returns file size  
in bytes
```

- **Closing a File**

- `fclose($f)` closes the file after operations are done.
 - Example:

PHP

```
fclose($f); // Closes the file
```

Files with Forms in PHP

Uploading Files to the Server

- **Form Setup for File Upload:**

- Use `<input type="file">` for selecting a file.
- Form must use the **POST** method and include `enctype="multipart/form-data"` to handle file uploads.
 - Example:

HTML

```
<form action="upload.php" method="POST"  
enctype="multipart/form-data">  
  <input type="file" name="f1">  
  <input type="submit" value="Upload">  
</form>
```

Temporary File Storage

- **Temporary Directory:**
 - The uploaded file is stored in a temporary directory as specified by `upload_tmp_dir` in `php.ini`.
 - The file is given a randomly generated name and will be lost if not processed by the script immediately.

Accessing Uploaded File Information

- **\$_FILES Superglobal:**
 - The `$_FILES` array holds details about the uploaded file.
 - The array has two levels:
 - The **first key** refers to the file input field name.
 - The **second key** provides details like file name, MIME type, size, temporary name, and error code.

File Information Available in \$_FILES

- `$_FILES["f1"]["name"]`: Original name of the file on the client's system.
- `$_FILES["f1"]["type"]`: MIME type of the file (e.g., "image/jpeg").
- `$_FILES["f1"]["size"]`: Size of the file in bytes.
- `$_FILES["f1"]["tmp_name"]`: Temporary file name on the server.
- `$_FILES["f1"]["error"]`: Error code related to file upload (e.g., `UPLOAD_ERR_OK`).

Saving the Uploaded File

- **Move and Rename File:**
 - Use `move_uploaded_file()` to move the file from the temporary directory to its final destination and rename it if needed.
 - Syntax: `move_uploaded_file($_FILES["f1"]["tmp_name"], $final_file_name)`
 - Returns `TRUE` if successful, otherwise `FALSE`.
 - Example:

PHP

```
if (move_uploaded_file($_FILES["f1"]["tmp_name"],
"uploads/new_name.jpg")) {
    echo "File uploaded successfully!";
} else {
    echo "File upload failed!";
}
```

Handling Multiple Form Inputs

- **Checkboxes and Multi-Selection Lists:**

- For fields like checkboxes or multi-selection lists, use ☐ in the field name to handle multiple selected values.
- In the form:

HTML

```
<input type="checkbox" name="option[]" value="1">
<input type="checkbox" name="option[]" value="2">
```

- In PHP, retrieve the values as an array:

PHP

```
$first_option = $_POST["option"][0]; // First selected
value
$second_option = $_POST["option"][1]; // Second selected
value
```

Notes on Cookies in PHP

Overview of Cookies

- **What are Cookies?**

- Small files stored on the client's machine (browser).
- Size limit: 4 KB per cookie.
- A server can store up to 20 cookies per client.
- Cookies can be disabled by the user.

- Common purposes include saving data, ensuring consistent user identification, and tracking for advertising.

Setting Cookies in PHP

- **Function to Set Cookies:**

- `setcookie(c_name, c_value, end_date, path, domain, security);`
- **c_name (Required):** The name of the cookie.
- **c_value:** The value associated with the cookie.
- **end_date:** Timestamp when the cookie expires (e.g., `time() + 86400` for 24 hours).
- **path:** Path within the website where the cookie is valid.
- **domain:** Specifies the domain the cookie is valid for, useful for cross-site cookies.
- **security:** Set to `TRUE` if cookie should be sent over secure HTTPS connections.

- **Example of Creating Cookies:**

PHP

```
setcookie("user", "JohnDoe", time() + 3600); // Cookie
expires in 1 hour
```

- **Deleting Cookies:**

- Set the cookie expiration date to a past timestamp to delete it:

PHP

```
setcookie("user", "", time() - 3600); // Deletes the cookie
```

Passing Multiple Values in a Cookie

- You can pass multiple values by using an array within a cookie:

PHP

```
setcookie("student[name]", "BENOMAR", time() + 3600);
setcookie("student[firstname]", "ALI", time() + 3600);
setcookie("student[faculty]", "FEI", time() + 3600);
```

Reading Cookies

- **Access Cookies via `$_COOKIE`:**
 - To retrieve a single value:

```
$student_name = $_COOKIE["student"]["name"];
```

PHP

- To access multiple values stored in the same cookie, use a **foreach** loop:

```
foreach ($_COOKIE["student"] as $key => $value) {  
    echo $key . ": " . $value . "<br>";  
}
```

PHP

Sessions in PHP:

Overview:

- **Stateless Nature of HTTP:** HTTP is stateless, meaning the server doesn't know if two requests come from the same client.
- **Sessions Introduced in PHP4:** The session mechanism was introduced to maintain state and track users across multiple pages.

Session Basics:

- **Session ID:** When a user visits a page, PHP generates a unique session ID. This ID can be passed through cookies or as part of the URL.
- **Session Variables:** These variables are stored on the server and are accessible across different pages during the session.
- **Session Life Cycle:**
 - The session starts when a unique identifier is generated.
 - Session variables are defined and stored in `$_SESSION`.
 - The session can be closed or destroyed to end the session.

Session Management:

1. Starting a Session:

Use `session_start();` at the beginning of each page to initialize the session.

2. Setting Session Variables:

- Use the `$_SESSION` array to store values:

```
$_SESSION['key'] = $value;
```

PHP

3. Accessing Session Variables:

- Access session data using `$_SESSION['key']`:

```
echo $_SESSION['key'];
```

PHP

4. Destroying Session Variables:

- To remove session variables:

```
session_unset();
```

PHP

- To destroy the session completely, including the session ID:

```
session_destroy();
```

PHP

Session Functions:

- **session_start():** Starts or resumes a session.
- **\$_SESSION:** Array used to store session variables.
- **session_unset():** Removes all session variables.
- **session_destroy():** Destroys the session completely.

Important Points:

- **Session Cookie:** By default, PHP sends the session ID in a cookie (**PHPSESSID**).
- **Security Considerations:** Sessions can be hijacked if not handled properly. Always use secure connections (HTTPS) and validate session data carefully.

****Object-Oriented PHP**

1. Declaring a Class

- Use the **class** keyword.
- Classes contain properties and methods.

PHP

```
class User {  
    public $Name;  
    public $Age;  
    function Print_Name() {  
        return $this->Name;  
    }  
}
```

2. Creating an Object

- Use the **new** keyword to instantiate a class.
- Access properties/methods with **->**.

PHP

```
$A_User = new User;  
$A_User->Name = "Karim";  
echo $A_User->Print_Name();
```

Constructors and Destructors

3. Constructor

- Special method: **__construct()**. Runs when an object is created.


```
class User {  
    function __construct($name) {  
        $this->Name = $name;  
    }  
}  
$A_User = new User("Amel");
```

4. Destructor

- Special method: `__destruct()`. Runs when an object is destroyed.

```
class User {  
    function __destruct() {  
        echo "User {$this->Name} is terminated.";  
    }  
}
```

Properties and Methods

5. Writing Methods

- Use `$this->property` to access object properties inside methods.
- Call methods with `$object->method()`.

6. Property Scope

- **public**: Accessible everywhere.
- **protected**: Accessible within the class and subclasses.
- **private**: Accessible only within the class.

Encapsulation with Getters and Setters

- Use `__get()` and `__set()` for controlled access to private properties.

```
class User {  
    private $Age;  
    function __get($name) {  
        return $this->$name;  
    }  
    function __set($name, $value) {  
        if ($name == "Age" && $value < 18) {  
            throw new Exception("Age must be 18+.");  
        }  
        $this->$name = $value;  
    }  
}
```

Static Members

7. Static Methods

- Called on the class, not objects. Use `ClassName::method()`.
- No access to `$this`.

8. Static Properties

- Declared with the `static` keyword.
- Accessed with `ClassName::$property`.

Inheritance

9. Basics

- Use `extends` for inheritance.
- Protected members are inherited but private ones are not.

```
class Customer extends User {  
    function display() {  
        $this->func_1();  
    }  
}
```

10. Overriding

- Subclasses can redefine properties and methods of parent classes.

11. Parent Keyword

- Use `parent::method()` to call the parent class's method.

```
class Customer extends User {  
    function func_1() {  
        parent::func_1();  
    }  
}
```

12. Final Keyword

- `final` prevents overriding methods or inheriting classes.

```
final class User { ... }
```

Traits (Simulating Multiple Inheritance)

- Use the `trait` keyword.
- Combine multiple traits in a class.
- Handle conflicts with `insteadof` or `as`.

```
trait Trait1 {  
    function method() { echo "Trait1"; }  
}  
trait Trait2 {  
    function method() { echo "Trait2"; }  
}  
class User {  
    use Trait1, Trait2 {  
        Trait1::method insteadof Trait2;  
        Trait2::method as method2;  
    }  
}
```

Best Practices

- Declare properties explicitly.
- Use visibility keywords (**public**, **protected**, **private**).
- Encapsulate access using getters and setters.
- Avoid direct access to static properties or methods unless necessary.

BY Habiba.